

ספר המערכת - לומדת ענף 71

מדריך המשתמש, המנהל והמפתח

תוכן עניינים

1. [מבוא](#)
 2. [התחלה מהירה](#)
 3. [ארכיטקטורה וטכנולוגיות](#)
 4. [טכנולוגיות בפירוט](#)
 5. [מדריך התלמיד](#)
 6. [מדריך המנהל](#)
 7. [עורך טקסט עשיר](#)
 8. [ניהול קבצים](#)
 9. [מדריך מפתח](#)
 10. [שתלשלות החלטות](#)
 11. [שאלות נפוצות](#)
 12. [נספחים](#)
-

מבוא

מה זו לומדת ענף 71?

לומדת ענף 71 היא מערכת למידה דיגיטלית מודרנית ומלאה לניהול קורסים, פרקים, וחומרי לימוד.

עבור תלמידים:

- צפייה בקורסים מסודרים 🧑
- קריאת פרקים עם תוכן עשיר (טקסט, תמונות, קבצים) 📖
- מעקב התקדמות בקורסים 📊
- פתרון בחינות וקבלת ציונים ✅
- הורדת חומרי לימוד (PDF, Word, Excel וכו') 📄

עבור מנהלים:

- יצירה וניהול קורסים מלא 📁
- כתיבת תוכן עם עורך טקסט WYSIWYG 📝
- העלאת תמונות וקבצים 🖼️
- יצירה וניהול של בחינות ושאלות ?
- ניהול משתמשים וגישות 👤
- איפוס סיסמה לתלמידים ומנהלים 🔑
- דשבורד ניהולי עם תמונת מצב כיתתית 📊

תכונות כלליות:

- עיצוב Udemy-style מודרני ✨
- מצב יום/לילה (Light/Dark) 🌙
- תמיכה מלאה בעברית (RTL) 🇮🇱
- Responsive design - עובד על כל המכשירים 📱
- אבטחה - הצפנת סיסמאות, access control 🔒

התחלה מהירה

שלב 1: דרישות מקדימות

Hardware:

- מחשב עם Windows/Mac/Linux 🖥️
- 4GB RAM מינימום 📀
- חיבור אינטרנט (להורדת dependencies) 🔗

Software:

- 18 Node.js + (<https://nodejs.org/>)
- 14 PostgreSQL + (<https://www.postgresql.org/>)
- דפדפן עדכני (Chrome, Firefox, Edge)
- Code Editor (VS Code, WebStorm וכו')

שלב 2: הפעלת המערכת**1. פתח את תיקיית הפרויקט:**

```
C:\Users\yairk\OneDrive\העבודה\rf-learning-system
```

2. לחץ פעמיים על קובץ `START.bat`

- שני חלונות CMD יופעלו (Frontend ו-Backend)
- הדפדפן יפתח אוטומטית ל-<http://localhost:5173>

3. המתן 15 שניות לשרתים להתחיל**שלב 3: התחברות**

בעמוד ההתחברות, בחר את התפקיד שלך:

אם אתה תלמיד:

```
123456789      : ת"ז
password123    : סיסמה
```

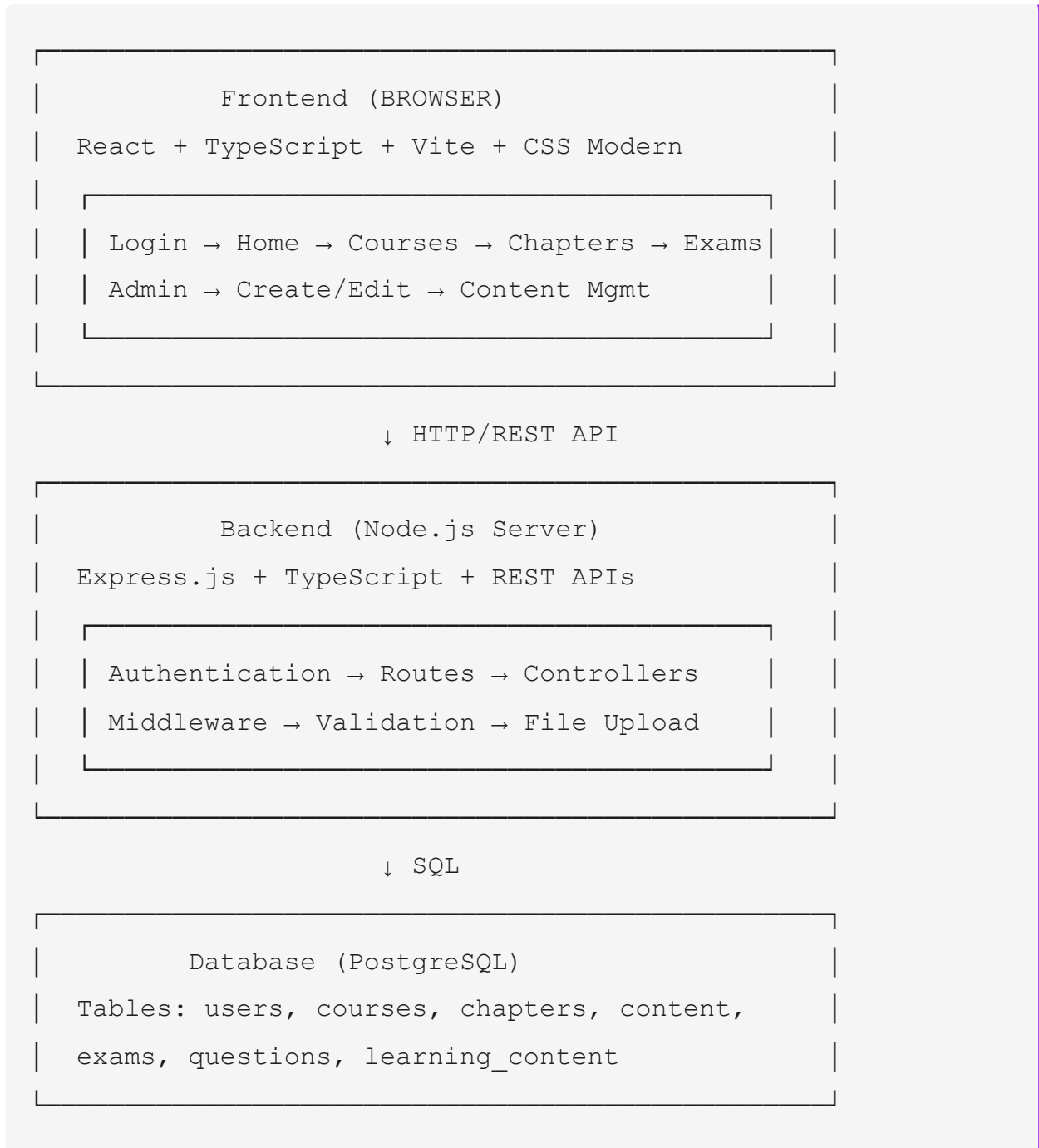
אם אתה מנהל:

```
999999999      : ת"ז
postgres123     : סיסמה
```

ארכיטקטורה וטכנולוגיות

סקירת המערכת

לומדת ענף 71 בנויה בארכיטקטורת **Tier-3 (שלוש שכבות)**:



Stack טכנולוגי (סקירה כללית) 🛠️

שימוש	גרסה	טכנולוגיה	Layer
UI components	x.18	React	Frontend
Type safety	5.9	TypeScript	Frontend
Build tool	8.1	Vite	Frontend
Styling, Grid, Flexbox	Modern	CSS3	Frontend
Navigation	x.6	React Router	Frontend
Runtime	+16	Node.js	Backend
Web framework	x.4	Express.js	Backend
Type safety	5.9	TypeScript	Backend
Development	2.0	ts-node-dev	Backend
Relational DB	+12	PostgreSQL	Database
Client library	x.8	pg	Database
File handling	1.4	Multer	File Upload
Hebrew support	Custom	Heebo	Font

טכנולוגיות בפירוט

Frontend Technologies 🎨

React 18.x

מה זה? ספריית JavaScript לבניית UI עם components reusable.

למה בחרנו?

- Component-based architecture
- Virtual DOM = performance
- Large community & ecosystem
- Perfect for single-page applications (SPA)

איך זה עובד?

```
// Component example
function CoursePage({ courseId }) {
  const [chapters, setChapters] = React.useState([]);

  React.useEffect(() => {
    fetchChapters(courseId).then(setChapters);
  }, [courseId]);

  return (
    <div className="course">
      {chapters.map(ch => <Chapter key={ch.id} data={ch} />)}
    </div>
  );
}
```

תכונות שבהן השתמשנו:

- Hooks (useState, useEffect, useContext)
- Functional components
- Context API לניהול state global

TypeScript 5.9

מה זה? JavaScript עם type checking.

למה בחרנו?

- Catch errors at compile time
- Better IDE autocomplete
- Self-documenting code

Easier refactoring •

דוגמה:

```
interface Course {
  id: number;
  title: string;
  description: string;
  createdAt: Date;
}

function displayCourse(course: Course): void {
  console.log(course.title); // ✅ TypeScript knows it's a string
  // console.log(course.price); // ❌ Error: property doesn't exist
}
```

Vite 8.1

מה זה? Build tool מהיר שמחליף Webpack.

למה בחרנו?

- Lightning fast dev server
- Native ES modules support
- Instant HMR (Hot Module Replacement)
- Optimized production builds

:Configuration

```
// vite.config.ts
import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    strictPort: false
  }
})
```

CSS3 Modern (No CSS framework)

מה זה? Native CSS עם Grid, Flexbox, variables.

למה בחרנו?

- No dependency on Bootstrap/Tailwind
- Full control over styling
- Custom design system
- Lighter bundle size

:CSS Variables System


```

:root {
  --bg: #ffffff;
  --text: #1c1d1f;
  --primary: #a435f0;
  --border: #d1d7dc;
  --success: #1e8e3e;
}

[data-theme="dark"] {
  --bg: #1c1d1f;
  --text: #f0f0f0;
  --primary: #c088f5;
}

body {
  background-color: var(--bg);
  color: var(--text);
}

```

React Router 6.x

מה זה? Navigation library לפיצול עמודים (pages) ללא page reload.

:Routes

```

const routes = [
  { path: '/', element: <HomePage /> },
  { path: '/login', element: <LoginPage /> },
  { path: '/course/:id', element: <CoursePage /> },
  { path: '/chapter/:id', element: <ChapterPage /> },
  { path: '/admin', element: <ProtectedRoute><AdminPage /></ProtectedRoute> },
];

```

Heebo Font

מה זה? Google Font עברי מודרני.

מדוע בחרנו?

- Perfect for Hebrew text
- Open source
- Multiple weights (400, 500, 700, 800, 900)
- Clean & modern look

Backend Technologies

+Node.js 16

מה זה? JavaScript runtime for server-side.

למה בחרנו?

- JavaScript on both frontend & backend (full-stack JS)
- Event-driven architecture
- Perfect for I/O heavy applications
- Excellent for real-time features

Express.js 4.x

מה זה? Minimalist web framework for Node.js.

למה בחרנו?

- Lightweight & flexible
- Easy middleware system
- Perfect for REST APIs
- Large ecosystem

:Example Route

```
// routes/courses.ts
router.get('/', async (req, res) => {
  try {
    const courses = await db.query('SELECT * FROM courses');
    res.json(courses.rows);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

router.post('/', authenticateAdmin, async (req, res) => {
  const { title, description } = req.body;
  const result = await db.query(
    'INSERT INTO courses (title, description) VALUES ($1, $2) RETURNING *'
  );
  res.status(201).json(result.rows[0]);
});
```

ts-node-dev 2.0

מה זה? Development tool שמריץ TypeScript בזמן אמת עם restart אוטומטי.

מדוע בחרנו?

- No build step needed
- Auto-restart on file changes
- Fast transpilation
- Perfect for development

npm script:

```
{
  "scripts": {
    "dev": "ts-node-dev --respawn --transpile-only src/server.ts"
  }
}
```

Multer 1.4

מה זה? Middleware להעלאת קבצים.

למה בחרנו?

- Standard library for file uploads
- Supports multipart/form-data
- Easy file validation
- Handles large files efficiently

:Configuration

```
// config/upload.ts
const upload = multer({
  dest: 'uploads/',
  fileFilter: (req, file, cb) => {
    const allowedExt = /\.(jpe?g|png|pdf|docx?|xlsx?|pptx?|csv|tiff)$/i;
    if (allowedExt.test(file.originalname)) {
      cb(null, true);
    } else {
      cb(new Error('File type not supported'));
    }
  },
  limits: { fileSize: 100 * 1024 * 1024 } // 100 MB
});
```

Database Technologies

+PostgreSQL 12

מה זה? Relational database system חזק ומוכשל.

למה בחרנו?

- ACID compliance
- Complex queries support
- Great for structured data
- Open source & stable
- Perfect for educational platforms

:Key Features

- Relationships (Foreign Keys)
- Transactions
- Indexes
- JSON support

:Tables Schema

```

-- Users
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    id_number VARCHAR(20) UNIQUE,
    password_hash VARCHAR(255),
    role VARCHAR(20), -- 'admin' or 'student'
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Courses
CREATE TABLE courses (
    id SERIAL PRIMARY KEY,
    title VARCHAR(200),
    description TEXT,
    created_by INT REFERENCES users(id),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Chapters
CREATE TABLE chapters (
    id SERIAL PRIMARY KEY,
    course_id INT REFERENCES courses(id) ON DELETE CASCADE,
    title VARCHAR(200),
    description TEXT,
    sort_order INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Learning Content (text, images, files)
CREATE TABLE learning_content (
    id SERIAL PRIMARY KEY,
    chapter_id INT REFERENCES chapters(id) ON DELETE CASCADE,
    type VARCHAR(50), -- 'TEXT', 'RICH', 'IMAGE', 'FILE'
    content TEXT,

```

```

    file_path VARCHAR(500),
    sort_order INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Exams
CREATE TABLE exams (
    id SERIAL PRIMARY KEY,
    course_id INT REFERENCES courses(id) ON DELETE CASCADE,
    title VARCHAR(200),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Questions
CREATE TABLE questions (
    id SERIAL PRIMARY KEY,
    exam_id INT REFERENCES exams(id) ON DELETE CASCADE,
    text VARCHAR(500),
    type VARCHAR(50), -- 'SINGLE', 'MULTIPLE', 'FREE_TEXT'
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Exam Results
CREATE TABLE exam_results (
    id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(id),
    exam_id INT REFERENCES exams(id),
    score INT,
    completed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

pg 8.x (PostgreSQL Client)

מה זה? PostgreSQL Node.js client library.

מדוע בחרנו?

- Pure JavaScript (no native dependencies)
- Supports prepared statements (prevents SQL injection)
- Connection pooling
- Promise-based

:Usage

```
import { Pool } from 'pg';

const pool = new Pool({
  host: process.env.DB_HOST,
  port: parseInt(process.env.DB_PORT),
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME
});

// Safe query with prepared statements
const result = await pool.query(
  'SELECT * FROM courses WHERE id = $1',
  [courseId]
);
```

Security Technologies

bcrypt

מה זה? Password hashing library

מדוע בחרנו?

- Industry standard
- Slow by design (prevents brute force)
- Salt generation automatic
- Proven secure

:Usage


```
import bcrypt from 'bcrypt';

// Hashing password
const salt = await bcrypt.genSalt(10);
const hashedPassword = await bcrypt.hash(userPassword, salt);

// Comparing password
const isMatch = await bcrypt.compare(userPassword, hashedPassword);
```

JWT (JSON Web Tokens)

מה זה? Stateless authentication tokens

מדוע בחרנו?

- Stateless = scalable
- Self-contained (no server session needed)
- Standard & widely supported
- Perfect for REST APIs

:Auth Flow

```
1. User logs in with ID + Password
   ↓
2. Server validates & creates JWT
   ↓
3. Client stores token in localStorage
   ↓
4. Every API request includes:
   Headers: { Authorization: "Bearer <token>" }
   ↓
5. Server verifies token with middleware
```

מדריך התלמיד

כמה תלמיד משתמש בלומדה 🎓

צעד 1: התחברות

1. הזן ת"ז וסיסמה
2. לחץ "התחברות"
3. תופיע דף הבית עם רשימת קורסים

צעד 2: בחירת קורס

בדף הבית תראה רשימה של קורסים.

כל קורס מציג:

-  שם הקורס
-  תיאור קצר
-  כפתור "התחל"

צעד 3: צפייה בקורס ופרקים

בדף הקורס תראה:

עמודה ימנית (רשימת פרקים):

-  רשימה של כל פרקי הקורס
-  סטטוס של כל פרק
-  מספר הפרק

עמודה שמאלית (תוכן קורס):

-  כותרת הקורס
-  תיאור הקורס
-  כרטיס התקדמות

צעד 4: קריאת פרק

לחץ על פרק מהרשימה.

בדף הפרק תראה:

- **כותרת הפרק** 📖
- **תיאור עם עיצוב** 📄
- **תמונות** 🖼️
- **קבצים להורדה** 📎
- **כפתורי ניווט** ➡️

צעד 5: פתרון בחינות

אם יש בחינות:

1. לחץ על **"בחינות"**
2. בחר **בחינה**
3. **ענה על השאלות**
4. לחץ **"שלח"**
5. קבל **ציון**

מדריך המנהל

ניהול מערכת 🧑

הפעלת פאנל ניהול

לאחר התחברות כמנהל:

1. לחץ על **"ניהול"** בתפריט
2. בחר:

- **קורסים** - יצירה/עריכה/מחיקה 📁
- **פרקים** - הוספה/עריכה/מחיקה 📖
- **תוכן** - הוספת טקסט, קבצים 📄
- **שאלות** ? - ניהול בחינות

יצירת קורס

שלבים:

1. בחר **"קורסים"** → **"הוסף קורס"**

2. מלא:

- **שם קורס** (חובה)
- **תיאור** (חובה)

3. לחץ **"שמור"**

הוספת פרק

שלבים:

1. בחר **קורס**

2. לחץ **"הוסף פרק"**

3. מלא:

- **שם הפרק** (חובה)
- **תיאור** (עורך WYSIWYG)
- **סדר** (מספר)

4. לחץ **"שמור"**

הוספת תוכן

שלבים:

1. בחר **פרק**

2. לחץ **"הוסף תוכן"**

3. בחר **סוג:**

-  **טקסט עשיר**
-  **תמונה**
-  **קובץ**

4. מלא וחשב **"שמור"**

דשבורד ניהולי

הדשבורד נותן למנהל תמונת מצב מרוכזת של הכיתה במסך אחד.

כניסה: בפאנל הניהול לחץ על **"דשבורד"** (או גש ישירות ל- `admin/dashboard/`).

מה מוצג:

- מספר התלמידים והמנהלים במערכת
 - מספר הקורסים והמגמות
 - ממוצע השלמת פרקים לתלמיד
 - ציון ממוצע בבחינות
 - **מבחנים הממתינים לבדיקה ידנית** (מודגש אם יש כאלה)
 - טבלת חמשת הקורסים עם הציון הממוצע הנמוך ביותר (לזיהוי חומר קשה)
- הנתונים מחושבים בזמן אמת מתוך בסיס הנתונים — אין צורך בעדכון ידני.

🔑 איפוס סיסמה למשתמש

כאשר תלמיד או מנהל שכח את סיסמתו, המנהל יכול לאפס אותה.

שלבים:

1. בפאנל הניהול לחץ על **"ניהול משתמשים"**
2. מצא את המשתמש ברשימה (תלמידים או מנהלים)
3. לחץ על **"🔑 אפס סיסמה"** ליד שמו
4. הזן סיסמה חדשה — חייבת לכלול אות גדולה, אות קטנה וספרה, באורך 6–8 תווים
5. הסיסמה נכנסת לתוקף **מיד**, והמשתמש יכול להתחבר איתה

הערה: איפוס הסיסמה פשוט ומיידי — המשתמש אינו נדרש להחליף אותה שוב בכניסה הבאה.

עורך טקסט עשיר

מה זה WYSIWYG?

WYSIWYG = "What You See Is What You Get" — עורך שמציג עיצוב בזמן אמת. משתמש ב-**contentEditable API** של HTML5, בלי ספריות חיצוניות.

כלי הסרגל

סמל	שם	דוגמה
B	Bold	טקסט בולד
I	Italic	טקסט אטליק
U	Underline	קו תחתון
H	Heading	### כותרת
	Color	צבע
	Unordered	• פריט 1
.1	Ordered	1. פריט 1
	Align Left	← יישור
	Align Right	יישור →
 	Align Center	→ יישור ←
X	Clear	טקסט רגיל

ניהול קבצים

סוגים נתמכים

סוג	סיומות	גודל
PDF	pdf.	MB 100
Word	doc, .docx .	MB 100
Excel	xls, .xlsx .	MB 100
PowerPoint	ppt, .pptx .	MB 100
טקסט	txt.	MB 100
CSV	csv.	MB 100

מדריך מפתח

התקנה ראשונה

```
# Backend setup
cd backend
npm install

# Frontend setup
cd ../frontend
npm install

# Setup Database
# ריק: rf_learning בשם DB צור
# דנית - המיגרציות רצות אוטומטית בעליית השרת SQL אין צורך להריץ
```

להתקנה מלאה על שרת production ראה את הקובץ `DEPLOY.md` בשורש הפרויקט.

npm Scripts

```
# Development
npm run dev                # Run with auto-restart

# Production
npm run build              # Build for production
npm start                 # Run production server

# Migrations (ידוני – בדרך כלל רץ אוטומטית)
npm run migrate            # הרצת מיגרציות שטרם הורצו
```

מנגנון מיגרציות אוטומטי

כדי למנוע חוסר-התאמה בין הקוד לבין סכמת בסיס הנתונים, המערכת מריצה את המיגרציות אוטומטית בכל עליית שרת:

- הקובץ `backend/src/db/migrate.ts` סורק את `database/*.sql` לפי סדר אלפביתי.
- טבלת מעקב `schema_migrations` רושמת אילו קבצים כבר הורצו, כך שכל קובץ רץ פעם אחת בלבד.
- מנגנון baseline:** על בסיס נתונים קיים (שהוקם ידנית בעבר), הקבצים הקיימים מסומנים כ"הורצו" בלי להריץ אותם מחדש — כדי למנוע כפילות.
- אם מיגרציה נכשלת — השרת **מסרב לעלות**, כדי שלא יעבוד על סכמה שבורה.

הוספת מיגרציה חדשה: צור קובץ חדש ב- `/database` בשם ממוין (למשל `migration_stage7_xxx.sql`), והוא ירוץ אוטומטית בעלייה הבאה.

מנהל ראשוני (Bootstrap Admin)

על שרת נקי טבלת המשתמשים ריקה. המיגרציה

יוצרת אוטומטית **מנהל זמני** — אך ורק `migration_stage6_bootstrap_admin.sql`

אם אין עדיין אף מנהל במערכת:

שם משתמש: OA0000000000

סיסמה: Admin123 (!זמנית – חובה להחליף)

הזרימה הנכונה: מתחברים עם המנהל הזמני → יוצרים מנהל OA אמיתי → **מוחקים את**

המנהל הזמני. ראה פירוט מלא ב- `DEPLOY.md`.

Directory Structure

```

rf-learning-system/
├── backend/
│   ├── src/
│   │   ├── controllers/      # Business logic
│   │   ├── routes/          # API routes
│   │   ├── middleware/      # Auth, validation
│   │   ├── config/          # Configuration
│   │   ├── db/              # Migration runner (migrate.ts)
│   │   └── server.ts         # Entry point
│   ├── uploads/             # Uploaded files
│   ├── .env.example         # Env template
│   └── package.json
│
├── database/                 # SQL schema + migrations (מאוחסן)
│
├── frontend/
│   ├── src/
│   │   ├── components/      # Reusable UI
│   │   ├── pages/           # Full pages
│   │   ├── context/         # Global state
│   │   ├── services/        # API calls
│   │   ├── App.tsx          # Root
│   │   └── App.css           # Styles
│   ├── dist/                # Build output
│   └── package.json
│
├── screenshots/             # Documentation
├── START.bat                 # Automation
└── SYSTEM_BOOK_FINAL.pdf     # This document

```

API Examples

Login

```
POST /api/auth/login
{ "id_number": "123456789", "password": "password123" }
Response: { "token": "...", "user": {...} }
```

Get Courses

```
GET /api/courses
Headers: { "Authorization": "Bearer <token>" }
Response: [{ "id": 1, "title": "...", ... }]
```

Create Course (Admin)

```
POST /api/courses
Headers: { "Authorization": "Bearer <admin_token>" }
Body: { "title": "...", "description": "..." }
```

שתלשלות החלטות

מדוע בחרנו React ולא Vue/Angular? 🤔

קריטריון	React	Vue	Angular
Learning curve	Middle	Easy	Hard
Performance	Excellent	Excellent	Good
Ecosystem	Huge	Good	Large
Community	Massive	Growing	Mature
For this project	Best choice ✅	Overkill ❌	Too heavy ❌

החלטה: React מעניק balance בין simplicity ל-power.

מדוע TypeScript ולא JavaScript? 🤔

:TypeScript

- Type safety at compile time ✓
- Better IDE support ✓
- Self-documenting code ✓
- Easier refactoring ✓
- Extra build step ✗
- Learning curve ✗

הבחירה: עבור מערכת TypeScript, large-scale, הוא חובה.

מדוע Vite ולא Webpack? 🤔

Webpack	Vite	Criteria
⚡ ⚡	⚡ ⚡ ⚡	Dev speed
Complex	Simple	Config
Hard	Easy	Learning
Proven	Recent	Maturity

החלטה: Vite = faster development experience.

מדוע PostgreSQL ולא MongoDB? 🤔

:PostgreSQL

- Relational structure ✓
- ACID compliance ✓
- Complex queries ✓
- Good for educational platforms ✓
- Well-known ✓

:MongoDB

- Flexible schema ✓ •
- Easy scaling ✓ •
- NoSQL = complex relationships ✗ •
- Overkill for this ✗ •

החלטה: PostgreSQL = perfect for structured learning data.

מדוע Express ולא NestJS/Fastify? 🤔

Fastify	NestJS	Express	Feature
Easy ✓	Complex ✗	Easy ✓	Simplicity
Low ✓	High ✗	Low ✓	Learning
Faster ✓ ✓	Great ✓	Good ✓	Performance
Overkill ✗	Overkill ✗	Perfect ✓	For MVP

החלטה: Express = lightweight, easy, perfect for MVP.

מדוע contentEditable ולא Quill/TinyMCE? 🤔

contentEditable (בחרנו):

- No dependencies ✓ •
- Lightweight ✓ •
- Full control ✓ •
- More complex to implement ✗ •

Quill/TinyMCE:

- Feature-rich ✓ •
- Proven ✓ •
- Large bundle ✗ •
- Complexity ✗ •

החלטה: contentEditable = לא צריכים features מורכבות, custom editor חוסך צרור.

?CSS Variables instead of SCSS/Tailwind 🤔

Tailwind	SCSS	CSS Vars	Option
Medium	Medium	Easy	Learning
Large	Small	Minimal	Bundle
Limited ❌	Great ✅	Great ✅	Customization
No ❌	No ❌	Yes ✅	Runtime theming
Overkill ❌	Overkill ⚠️	Perfect ✅	For us

החלטה: CSS Variables = exact control for Dark/Light themes.

שאלות נפוצות

? "איזה גרסת Node.js צריך?"

תשובה: Node.js 18 + (בדוק עם `node --version`)

? "Database rf_learning not found"

פתרון:

1. pgAdmin → Databases

2. Create → Database

3. שם: `rf_learning`

4. הפעל את השרת — המיגרציות ייצרו את הטבלאות אוטומטית

? "Port 3000/5173 already in use"

פתרון:

```
taskkill /F /IM node.exe
```

? "איך מוסיפים משתמש חדש?"

תשובה: דרך פאנל הניהול → "ניהול משתמשים" → הוספת משתמש. מנהל יכול גם לאפס סיסמאות משם.

? "השרת לא עולה ומציג שחסר משתנה סביבה"

תשובה: זו התנהגות מכוונת. הגדר את המשתנים החסרים בקובץ `backend/.env` (ראה תבנית ב- `env.example`) והפעל שוב.

? "איך מוציאים לProduction?"

תשובה: ראה מדריך מלא בקובץ `DEPLOY.md`. בקצרה צריך:

1. Server עם Node.js 18 ו-PostgreSQL
 2. הגדרת קובץ `env.` (כולל סודות JWT ו-`CORS_ORIGIN`)
 3. `npm run build` ואז `npm start` (המיגרציות רצות אוטומטית)
 4. הגשת `frontend/dist` דרך שרת ווב (Nginx) + SSL
 5. **החלפת המנהל הזמני** (ראה `DEPLOY.md`)
-

נספחים

נספח א': תוכן עיוני עמוק — מנועי בוכנה

מנוע בוכנה (Piston Engine) — מבוא מלא

1. עיקרון הפעולה

מנוע בוכנה הוא מנוע בעירה פנימית המשתמש בתהליך של:

ארבע פעימות:

1. Intake (יניקה):

- הבוכנה יורדת
- שסתום יניקה נפתח
- תערובת דלק + אויר נשאבת לתא הבעירה

2. Compression (דחיסה):

- הבוכנה עולה
- שתי שסתומים סגורים
- תערובת דלק + אויר דחוסה לכ-10:1

3. Power (בעירה):

- ניצוץ חשמלי מנקודת הנחת
- דלק בוער בחזקה
- גזים חמים דוחפים בוכנה למטה
- זה מייצר את הכוח!

4. Exhaust (פליטה):

- הבוכנה עולה שוב
- שסתום פליטה נפתח
- גזים שורופים יוצאים החוצה

2. יתרונות מנוע בוכנה

יתרונות קטגוריה 1: כלכלי

- עלות הייצור נמוכה משמעותית 💰
- דלק זול וקל להשגה בכל מקום 💰
- צעצועים יציא להחלפה (בלוקים, בוכנות וכו') 💰
- שכרי טכנאים נמוכים יותר 💰
- ניתן לתיקון בתחנות כפריות 💰

יתרונות קטגוריה 2: טכני

- עמידות גבוהה מאוד (עד 500,000 ק"מ) ⚙️
- פשטות ברשימת החלקים ⚙️
- ביצועים טובים בעומסים גבוהים ⚙️
- טווח טמפרטורה תפעולי רחב ⚙️
- ניתן להתאם לדלקים שונים ⚙️

יתרונות קטגוריה 3: חיזוק

- טוב לאקראות חלופיות 🛡️
- אינו דורש טכנולוגיה מתקדמת 🛡️
- ניתן לעבוד עם דלקים ישנים 🛡️

3. חסרונות מנוע בוכנה

חסרונות קטגוריה 1: סביבה

- זיהום גבוה של חמצן חד-תחמוצתי (CO) 🌍
- פליטות NOx מזיקות לשכבת האוזון 🌍
- ריח פליטה חזק 🌍
- שמע עליל מאוד (עד 90 dB) 🌍

חסרונות קטגוריה 2: תפעולי

- צריכת דלק גבוהה (10-5 ליטר/100 ק"מ) 🔧
- תחזוקה קבועה נדרשת 🔧
- עלויות ספירה (שמן, פילטר, בדיקות) 🔧
- קולחון מנוע דורש ניקוי תדיר 🔧

חסרונות קטגוריה 3: קושי בשימוש

- ייקום בימות קרות כ"כ 🥵

4. מפרט טכני ממוצע

פרמטר	ערך
נפח מנוע (CC)	1000-3000
מספר צילינדרים	2-8
הספק (HP)	50-300
מומנט מרבי (Nm)	100-400
RPM מרבי	5000-7000
יחס דחיסה	8:1-10:1
טמפרטורת פעולה	80-95°C
כיסוי שמן	4-6 ליטר

5. שרטוטי מנוע

קטעים עיקריים:

- Cylinder Block (בלוק צילינדר)
- Piston (בוכנה)
- Connecting Rod (מוט חיבור)
- Crankshaft (ציר ארכובה)
- Valve Train (מערכת שסתומים)
- Spark Plug (נקודת נחת)

נספח ב': תוכן עיוני עמוק — מנועי סילון

מנוע סילון (Jet Engine) — מבוא מלא

1. עיקרון הפעולה

מנוע סילון משתמש ב- **Brayton Cycle**:

4 שלבים עיקריים:

1. Intake (יניקה):

- אוויר חם נשאב בחזקה
- מעבר דרך diffuser

2. Compression (דחיסה):

- מדחס (Compressor) עם טורבינה מרובת דרגות
- אוויר דחוס ל-30:1 יחס

3. Combustion (בעירה):

- דלק זריקה לתא בעירה
- טמפרטורה עולה ל- 1500°C
- בעירה מתמשכת (לא פעימות!)

4. Exhaust (סילון):

- גזים חמים מאד יוצאים בעוצמה
- יוצרים דחף (thrust) מעצום

2. יתרונות מנוע סילון

יתרונות קטגוריה 1: ביצועים

- דחף עצום (עד $100,000 \text{ kN}$) 🚀
- מהירות גבוהה מאוד 🚀
- עומד בטמפרטורות קיצוניות 🚀
- ביצוע יציב בגבהים גבוהים 🚀

יתרונות קטגוריה 2: טכני

-  דיוק אווירודינמי שלם
-  לא צריך הילוך (ישיר דחף)
-  בעירה יציבה (לא פעימות)
-  חלקים נעים מעטים

יתרונות קטגוריה 3: תפעול

-  כושר הטסה גבוה ביותר
-  עמידות בתנאים קשים
-  מדיום גדול (מ-50 עד 1000 + ק"ג דלק)

3. חסרונות מנוע סילון

חסרונות קטגוריה 1: כלכלי

-  עלות ייצור אדירה
-  דלק מתוקצר (Jet Fuel)
-  טכנאים מיוחדים בלבד
-  תחזוקה מעצימה ויקרה

חסרונות קטגוריה 2: תפעולי

-  צריכת דלק ענקית (עד 5000 ליטר לשעה!)
-  קול עליל מעצום (עד 140 dB!)
-  פליטות גזים גבוהות
-  דרוש runway לכת וקליטה

חסרונות קטגוריה 3: בטיחות

-  סכנת בעירה מופרזת
-  דרוש קירור מיוחד
-  עומס מכני ענק

4. מפרט טכני ממוצע

פרמטר	ערך
יחס דחיסה	25:1-40:1
דחף (kN)	50-100
משקל	1000-2000 ק"ג
אורך	2-5 מטרים
טמפרטורת בעירה	1200-1600°C
צריכת דלק	2000-5000 ליטר/שעה
רוח יציאה	500-700 מ/שנ

נספח ג': פן תכנותי

Code Examples

Authentication Middleware

```

import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';

interface JWTPayload {
  userId: number;
  role: 'admin' | 'student';
}

export const authenticateToken = (
  req: Request,
  res: Response,
  next: NextFunction
) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) return res.sendStatus(401);

  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) return res.sendStatus(403);
    req.user = user as JWTPayload;
    next();
  });
};

export const authorizeAdmin = (
  req: Request,
  res: Response,
  next: NextFunction
) => {
  if (req.user.role !== 'admin') {
    return res.status(403).json({ error: 'Admin access required'
  }

```

```
next();  
};
```

RichTextEditor Component

```

import React, { useRef, useState } from 'react';

interface RichTextEditorProps {
  onChange: (html: string) => void;
  initialValue?: string;
}

export const RichTextEditor: React.FC<RichTextEditorProps> = ({
  onChange,
  initialValue = ''
}) => {
  const editorRef = useRef<HTMLDivElement>(null);

  const applyFormat = (command: string, value?: string) => {
    document.execCommand(command, false, value);
    editorRef.current?.focus();
  };

  const handleInput = () => {
    if (editorRef.current) {
      onChange(editorRef.current.innerHTML);
    }
  };

  return (
    <div className="rte">
      <div className="rte-toolbar">
        <button onClick={() => applyFormat('bold')}>B</button>
        <button onClick={() => applyFormat('italic')}>I</button>
        <button onClick={() => applyFormat('underline')}>U</button>
        <button onClick={() => applyFormat('createUnorderedList')}>
        <button onClick={() => applyFormat('createOrderedList')}>
        <button onClick={() => applyFormat('justifyCenter')}>X</button>
        <button onClick={() => applyFormat('removeFormat')}>X</button>
      </div>

```



```
<div
  ref={editorRef}
  className="rte-editor"
  contentEditable
  onInput={handleInput}
  dangerouslySetInnerHTML={{ __html: initialValue }}
/>
</div>
);
};
```

API Service (Frontend)

```

const API_URL = 'http://localhost:3000/api';

export const apiService = {
  // Authentication
  async login(idNumber: string, password: string) {
    const response = await fetch(`${API_URL}/auth/login`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ id_number: idNumber, password })
    });
    const data = await response.json();
    localStorage.setItem('token', data.token);
    return data.user;
  },

  // Courses
  async getCourses() {
    return this.fetch(`${API_URL}/courses`);
  },

  async createCourse(title: string, description: string) {
    return this.fetch(`${API_URL}/courses`, {
      method: 'POST',
      body: JSON.stringify({ title, description })
    });
  },

  // Helper
  private async fetch(url: string, options?: RequestInit) {
    const token = localStorage.getItem('token');
    const headers = {
      'Authorization': `Bearer ${token}`,
      ...options?.headers
    };
  };

```

```
const response = await fetch(url, { ...options, headers });
if (!response.ok) throw new Error(`HTTP ${response.status}`);
return response.json();
}
};
```

ניתוח השוואתי

מנוע בוכנה vs מנוע סילון

Aspect	בוכנה	סילון
דחף	נמוך	ענק
מהירות	עד 300 קמ"ש	עד 900+ קמ"ש
צריכה	נמוכה	גבוהה מאוד
עלות	זולה	יקרה
תחזוקה	קלה	מורכבת
סביבה	זיהום	יותר זיהום
שימוש	רכבים יבשתיים	מטוסים

סוף הספר

עדכון אחרון: 2026-06-28 גרסה: Final 1.0 שפות: TypeScript, JavaScript, SQL, HTML, CSS

סטטוס: Production Ready